Building RAG Apps and Agents on Azure AI Foundry

Azure AI Foundry guide, volume 2 · Version 2026.1

> Sample foundation document written for testing a domain-expert RAG. Azure AI Foundry evolves quickly and product names have changed over time; confirm details in the official Microsoft Learn documentation.

1. RAG architecture on Azure

Component	Azure service
Document storage	Azure Blob Storage or Data Lake
Parsing and OCR	Azure AI Document Intelligence (layout, tables, forms)
Chunking and embedding	Azure AI Search integrated vectorisation or your own pipeline with an embedding model deployment
Index and retrieval	Azure AI Search: keyword, vector, hybrid and semantic ranking
Generation	A chat model deployment in Foundry
Safety	Azure AI Content Safety and Foundry content filters
Evaluation and monitoring	Foundry evaluations, tracing and Azure Monitor / Application Insights

2. Azure AI Search for retrieval

Azure AI Search stores documents in an index with fields for text, vectors and metadata. **Hybrid search** runs keyword (BM25) and vector queries together and merges them with reciprocal rank fusion. The **semantic ranker** re-scores the top results with a language model and can return captions and answers. Filters on metadata fields enforce security trimming, for example returning only documents whose group field matches the caller's groups. Integrated vectorisation can chunk and embed data during indexing using indexers and skillsets.

3. Grounding a chat model on your data

1. Upload documents to storage and create a search index, with vector fields and metadata.
2. Deploy an embedding model and a chat model in the Foundry project.
3. Connect Azure AI Search to the project with managed identity.
4. In the playground or code, retrieve passages with hybrid search plus semantic ranking and pass them to the chat model with instructions to cite sources.
5. Apply filters for security trimming and add a content filter configuration to the deployment.
6. Evaluate groundedness and relevance before release and monitor them in production.

4. Prompt flow

Prompt flow is a development tool for LLM workflows. A **flow** is a graph of nodes: LLM calls, Python functions, prompts and tools. Variants let you compare prompt versions; batch runs execute a flow over a dataset; evaluation flows score the outputs. Flows can be deployed as online endpoints. Prompt flow is most associated with hub-based projects; newer agent and evaluation features are available directly in Foundry projects.

5. Azure AI Foundry Agent Service

The Foundry Agent Service hosts agents that combine a model, instructions and tools, and manages conversation state in threads so the application does not have to.

Tool	Use
File search	Retrieval over uploaded files with a managed vector store
Azure AI Search	Grounding on an existing enterprise index
Grounding with Bing Search	Current web information, with citations
Code interpreter	Run Python in a sandbox for analysis and charts
Function calling	Call your own code with typed arguments
OpenAPI tools	Call REST APIs described by an OpenAPI specification
Connected agents / multi-agent	Delegate sub-tasks to other agents

A run processes a thread: the agent decides which tools to call, the service executes

built-in tools, and your code handles function calls. Store secrets in Key Vault connections, give each agent's tools the least access they need, and require confirmation in your application for actions with side effects.

6. Tracing and monitoring

Foundry integrates with OpenTelemetry-based tracing, sending traces to Application Insights so you can see each request's prompts, tool calls, token counts and latency. Production monitoring can run evaluators on a sample of live traffic and alert when groundedness, relevance or safety scores drift.

7. Fine-tuning

Fine-tuning is available for selected Azure OpenAI and open models. Prepare training data in the chat JSONL format, hold out a validation set, and compare the fine-tuned model with the base model on your evaluation set before deploying. Fine-tune for format, style and narrow tasks; keep facts in RAG so they stay current.